

GymPro — Complete Code Guide

From Beginning to End — Every File, Every Line Explained

Spring Boot 3.x | Spring Cloud | JWT | MySQL | OpenFeign

9 Microservices: eureka • gateway • auth • member • trainer • plan • booking • notification • payment

Contents

#	Section	What it covers
1	What is this project?	Big picture — 9 services, what each does, how they talk
2	pom.xml	Maven dependencies — what libraries we imported and why
3	application.properties	Config file — port, database, JWT, Eureka settings
4	XxxApplication.java	The main() class — how Spring Boot starts up
5	SecurityConfig.java	Spring Security — why we disable it in most services
6	entity/ folder	@Entity classes — how Java maps to MySQL tables
7	repository/ folder	JpaRepository — free SQL methods, custom queries
8	service/ folder	Business logic — what happens when you hit an endpoint
9	controller/ folder	REST endpoints — @GetMapping, @PostMapping explained
10	dto/ folder	Request/Response objects — what JSON looks like
11	JwtUtil.java	How JWT token is built, signed, and read
12	JwtAuthFilter.java	Gateway security guard — runs on every request
13	NotificationClient.java	OpenFeign — how services call each other
14	Full request walkthrough	Postman → Gateway → Service → MySQL → back
15	Every annotation explained	@SpringBootApplication to @RequestParam

What is this project?

Big picture before we read a single line of code

Think of GymPro like a city with 9 different offices

Instead of building ONE giant application, we built 9 small, independent Spring Boot apps. Each app does one job. They run on different ports and have their own MySQL database. They talk to each other through HTTP (like websites calling each other).

Service	Port	Database	Its one job	Who calls it
eureka-server	8761	None	Phone book — every service registers its address here	Edgeworker startup
gateway-service	8080	None	Front door — every Postman request comes here first. Checks JWT token	Postman / browser
auth-service	8081	gympro_auth	Register users, check passwords, issue JWT tokens	gateway routes /auth/**
member-service	8082	gympro_member	Create/read/update/delete gym member profiles	gateway routes /members/**
trainer-service	8083	gympro_trainer	Trainer profiles + weekly schedule slots	gateway routes /trainers/**
plan-service	8084	gympro_plan	Membership plans + who subscribed to what	gateway routes /plans/**
booking-service	8085	gympro_booking	Book a trainer session. Sends email via Edgeworker	gateway routes /bookings/**
notification-service	8086	None	Send Gmail SMTP emails. Called by booking-service	gateway routes /booking/payment
payment-service	8087	gympro_payment	Dummy payment flow. Sends receipt email via Edgeworker	gateway routes /payments/**

Tip: Every service is a completely separate Spring Boot project with its own pom.xml, main class, and application.properties. If one service crashes, the others keep running.

What is pom.xml?

pom.xml is the Maven build file. It tells Maven (the build tool) which external libraries to download. Think of it as your project's shopping list. You write what you need, Maven downloads it from the internet.

Key dependencies and why we added each one:

<code>spring-boot-starter-web</code>	Used in: all business services Gives us <code>@RestController</code> , <code>@GetMapping</code> etc. Starts an embedded Tomcat web server automatically.
<code>spring-boot-starter-data-jpa</code>	Used in: all services with a DB Gives us <code>JpaRepository</code> , <code>@Entity</code> , <code>@Table</code> . Connects Java to MySQL using Hibernate.
<code>spring-boot-starter-security</code>	Used in: all services Spring Security framework. We mostly disable its defaults but need it for BCrypt password hashing.
<code>mysql-connector-j</code>	Used in: all services with a DB The JDBC driver — the wire between Java and MySQL. Without this, no database connection.
<code>spring-cloud-starter-netflix-eureka-client</code>	Used in: all except eureka-server Makes the service register with Eureka on startup so gateway can find it.
<code>spring-cloud-starter-netflix-eureka-server</code>	Used in: eureka-server only Turns the app into a Eureka registry server.
<code>spring-cloud-starter-gateway</code>	Used in: gateway-service only Spring Cloud Gateway — reactive routing, filters, load balancing.
<code>spring-cloud-starter-openfeign</code>	Used in: booking + payment services OpenFeign — lets you call another microservice by writing an interface, not HTTP code.
<code>spring-boot-starter-mail</code>	Used in: notification-service only <code>JavaMailSender</code> — connects to Gmail SMTP and sends emails.
<code>jjwt-api / jjwt-impl / jjwt-jackson</code>	Used in: auth-service + gateway JSON Web Token library — build and validate JWT tokens.
<code>lombok</code>	Used in: all services Annotation processor — auto-generates getters, setters, constructors at compile time.

Note: Each service has its own pom.xml with only the dependencies it needs. booking-service needs openfeign. notification-service needs mail. eureka-server needs neither.

What is application.properties?

This file lives in `src/main/resources/`. Spring reads it on startup. It's how you configure your app WITHOUT changing Java code. Port, database URL, passwords, JWT secret — all go here.

auth-service — application.properties (full explanation)

```
server.port=8081

spring.application.name=auth-service

spring.datasource.url=jdbc:mysql://localhost:3306/gympro_auth?createDatabaseIfNotE
xist=true

spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.instance.prefer-ip-address=true

jwt.secret=gympro_jwt_secret_key_for_hs256_algorithm_minimum_256_bits
jwt.expiration=86400000
```

<code>server.port=8081</code>	This service listens on port 8081. Each service must have a unique port.
<code>spring.application.name=auth-service</code>	The name Eureka uses to register this service. gateway uses <code>lb://auth-service</code> to find it.
<code>jdbc:mysql://localhost:3306/gympro_auth</code>	Connect to MySQL on your machine (localhost), port 3306, database named <code>gympro_auth</code> .
<code>createDatabaseIfNotExist=true</code>	MySQL will create <code>gympro_auth</code> database automatically if it doesn't exist yet.
<code>username=root / password=root</code>	MySQL login credentials. Must match your MySQL installation.
<code>ddl-auto=update</code>	Hibernate reads your <code>@Entity</code> classes and AUTO-CREATES or updates MySQL tables. No SQL needed.
<code>show-sql=true</code>	Print every SQL query Hibernate runs in the Eclipse console. Good for debugging.
<code>eureka.client.service-url.defaultZone</code>	Where to find Eureka. On startup, auth-service sends its address here so gateway can find it.
<code>jwt.secret</code>	The secret key used to sign JWT tokens. MUST be the same in auth-service AND gateway-service.

```
jwt.expiration=86400000
```

Token expires in 86400000 milliseconds = 24 hours. After this, user must login again.

Tip: Other services (member, trainer, plan, booking, payment) have the same datasource block but with DIFFERENT database names: gympro_member, gympro_trainer etc. notification-service has NO datasource — it only sends emails.

What is the main class?

Every microservice has exactly ONE main class with a `main()` method. When you click Run As → Spring Boot App in Eclipse, Java calls this `main()` method first. `SpringApplication.run()` then starts the entire Spring framework.

Three different main class patterns used in GymPro:

Pattern 1 — eureka-server (unique)

```
@SpringBootApplication
@EnableEurekaServer // <-- makes this app a registry server
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Pattern 2 — most business services (member, trainer, plan, auth, notification)

```
@SpringBootApplication
@EnableDiscoveryClient // <-- registers with Eureka on startup
public class MemberServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(MemberServiceApplication.class, args);
    }
}
```

Pattern 3 — booking + payment services (they call other services via Feign)

```
@SpringBootApplication
@EnableDiscoveryClient // <-- registers with Eureka
@EnableFeignClients // <-- scans for @FeignClient interfaces
public class BookingServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(BookingServiceApplication.class, args);
    }
}
```

`@SpringBootApplication` = `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan` combined. Starts everything.

<code>@EnableEurekaServer</code>	Only in eureka-server. Makes the app host the Eureka dashboard at port 8761.
<code>@EnableDiscoveryClient</code>	Makes the service send its name+address to Eureka on startup. Gateway then finds it by name.
<code>@EnableFeignClients</code>	Scans all packages for <code>@FeignClient</code> interfaces and creates HTTP proxy objects for them.
<code>SpringApplication.run(..)</code>	Bootstraps Spring context, starts Tomcat, registers with Eureka, opens port for requests.

Why do we need SecurityConfig?

Spring Security is included in every service's pom.xml. By default, it blocks ALL requests and shows a login page. We need to override this. There are TWO different SecurityConfig patterns in GymPro.

Config A — auth-service (special — needs BCrypt)

```
@Configuration
@EnableWebSecurity

public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // used to hash passwords
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // disable CSRF – not needed for REST
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll() // login/register: no token needed
                .anyRequest().authenticated() // everything else: needs token
            );
        return http.build();
    }
}
```

Config B — all other services (just disable defaults, gateway handles auth)

```
@Configuration
@EnableWebSecurity

public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(c -> c.disable())

        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(a -> a.anyRequest().permitAll()); // allow all – gateway checks

        return http.build();
    }
}
```



```
}
```

<code>@Configuration</code>	This class contains Spring <code>@Bean</code> definitions. Spring reads it at startup.
<code>@EnableWebSecurity</code>	Activates Spring Security and allows us to override its defaults.
<code>@Bean PasswordEncoder</code>	Creates a <code>BCryptPasswordEncoder</code> object managed by Spring. <code>AuthService</code> <code>@Autowired</code> s it to hash passwords.
<code>csrf.disable()</code>	CSRF protection is for browser form attacks. REST APIs use tokens, so CSRF is not needed here.
<code>SessionCreationPolicy.STATELESS</code>	No HTTP sessions. Every request must carry its own token. Spring doesn't remember previous requests.
<code>permitAll()</code>	In other services: let ALL requests through. The gateway already validated the JWT before routing here.
<code>/auth/** permitAll()</code>	In auth-service: login and register don't have a token yet, so they must be allowed without one.

What is an Entity?

An entity is a Java class that maps directly to a database table. Every field in the class becomes a column in MySQL. Hibernate (part of JPA) reads these annotations and creates the table automatically when `ddl-auto=update` is set.

Example: User.java (auth-service) → creates 'users' table in gympro_auth

```
@Entity // tells Hibernate: make a table for this class
@Table(name = "users") // the MySQL table will be named 'users'
@Data // Lombok: generates getters + setters + toString
@NoArgsConstructor // Lombok: empty constructor — JPA REQUIRES this
@AllArgsConstructor // Lombok: constructor with all fields
public class User {

    @Id // this field is the PRIMARY KEY
    @GeneratedValue(strategy = GenerationType.IDENTITY) // AUTO_INCREMENT in MySQL
    private Long id;

    private String name; // plain field → VARCHAR(255) column

    @Column(unique = true, nullable = false)
    private String email; // UNIQUE + NOT NULL constraint in MySQL

    @Column(nullable = false)
    private String password; // NOT NULL — stored as BCrypt hash

    @Column(nullable = false)
    private String role; // 'MEMBER' | 'TRAINER' | 'ADMIN'
}
```

MySQL table Hibernate creates from this:

```
CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255),
  email VARCHAR(255) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role VARCHAR(255) NOT NULL
);
```

All entity classes in GymPro:

Entity Class	Service	Database	Table	Key Fields
User.java	auth-service	gympro_auth	users	id, name, email, password, role

Member.java	member-service	gympro_member	members	id, name, email, phone, address, gender, status
Trainer.java	trainer-service	gympro_trainer	trainers	id, name, email, phone, specialization, experienceYear
TrainerSchedule.java	trainer-service	gympro_trainer	trainer_schedules	id, trainerId, dayOfWeek, startTime, endTime, availability
MembershipPlan.java	plan-service	gympro_plan	membership_plans	id, planName, description, durationType, price, duration
MemberSubscription.java	plan-service	gympro_plan	member_subscriptions	id, memberId, memberEmail, planId, planName, startDate
Booking.java	booking-service	gympro_booking	bookings	id, memberId, memberEmail, trainerId, trainerEmail, slotId
Payment.java	payment-service	gympro_payment	payments	id, memberId, memberEmail, bookingId, subscriptionId

What is a Repository?

A repository is just an interface that extends JpaRepository. You write almost nothing — Spring Data JPA generates all the SQL at runtime. You get save(), findById(), findAll(), delete() for FREE.

Example: MemberRepository.java

```
public interface MemberRepository extends JpaRepository {
    // FREE methods you get automatically (you don't write these):
    // save(member) → INSERT or UPDATE
    // findById(id) → SELECT * FROM members WHERE id=?
    // findAll() → SELECT * FROM members
    // deleteById(id) → DELETE FROM members WHERE id=?
    // existsById(id) → SELECT COUNT(*) FROM members WHERE id=?

    // Custom methods – Spring reads the name and generates SQL:
    Optional findByEmail(String email);
    // → SELECT * FROM members WHERE email=?
}
```

All custom repository methods in GymPro explained:

Repository	Method	Generated SQL / Purpose
UserRepository	findByEmail(email)	SELECT * FROM users WHERE email=? – used in login
UserRepository	existsByEmail(email)	SELECT COUNT(*) > 0 FROM users WHERE email=? – check
MemberRepository	findByEmail(email)	SELECT * FROM members WHERE email=?
TrainerRepository	findBySpecialization(spec)	SELECT * FROM trainers WHERE specialization=? – filter
TrainerRepository	findByStatus(status)	SELECT * FROM trainers WHERE status=? – get only ACTIVE
TrainerScheduleRepository	findByTrainerId(id)	SELECT * FROM trainer_schedules WHERE trainer_id=?
TrainerScheduleRepository	findByTrainerIdAndAvailable(id, true)	SELECT * FROM trainer_schedules WHERE trainer_id=? AND available
BookingRepository	findByMemberId(id)	SELECT * FROM bookings WHERE member_id=?
BookingRepository	findByTrainerId(id)	SELECT * FROM bookings WHERE trainer_id=?
BookingRepository	findByStatus(status)	SELECT * FROM bookings WHERE status=?
MembershipPlanRepository	findByActive(true)	SELECT * FROM membership_plans WHERE active=1 – only active
MemberSubscriptionRepository	findByMemberId(id)	SELECT * FROM member_subscriptions WHERE member_id=?
PaymentRepository	findByTransactionId(txn)	SELECT * FROM payments WHERE transaction_id=?
PaymentRepository	findByBookingId(id)	SELECT * FROM payments WHERE booking_id=?

Note: Spring Data JPA reads the method name like a sentence: findBy + FieldName + [And + FieldName]. It writes the SQL for you. You NEVER write 'SELECT * FROM ...' manually in this project.

What is a Service class?

The service layer is where all the actual work happens. The controller receives the HTTP request and immediately hands it to the service. The service uses the repository to talk to the database and returns the result.

AuthService.java — register() method fully explained

```
@Service
public class AuthService {

    @Autowired private UserRepository repo; // inject the repo
    @Autowired private JwtUtil jwtUtil; // inject JWT helper
    @Autowired private PasswordEncoder encoder; // inject BCrypt

    public AuthResponse register(User user) {

        // Step 1: Check if email already exists
        if (repo.existsByEmail(user.getEmail())) {
            throw new RuntimeException("Email already registered");
            // Spring catches this → returns HTTP 500 with message
        }

        // Step 2: Set default role if none given
        if (user.getRole() == null || user.getRole().isBlank()) {
            user.setRole("MEMBER");
        }

        // Step 3: Hash the password – NEVER store plain text
        user.setPassword(encoder.encode(user.getPassword()));
        // "admin123" becomes "$2a$10$N9qo8uL0i..."

        // Step 4: Save to MySQL → INSERT INTO users(...)
        User saved = repo.save(user);

        // Step 5: Create JWT token with email + role inside
        String token = jwtUtil.generateToken(saved.getEmail(), saved.getRole());

        // Step 6: Return JSON response
        return new AuthResponse(token, saved.getEmail(), saved.getRole(), "Registered successfully");
    }
}
```

AuthService.java — login() method fully explained

```

public AuthResponse login(LoginRequest req) {

    // Step 1: Find user by email
    User user = repo.findByEmail(req.getEmail())
        .orElseThrow(() -> new RuntimeException("User not found"));
    // orElseThrow: if Optional is empty (user not found) → throw exception

    // Step 2: Check if password matches the stored BCrypt hash
    if (!encoder.matches(req.getPassword(), user.getPassword())) {
        throw new RuntimeException("Wrong password");
    }

    // BCrypt re-hashes the input and compares – you cannot decrypt BCrypt

    // Step 3: Generate new JWT token
    String token = jwtUtil.generateToken(user.getEmail(), user.getRole());

    // Step 4: Return token to client
    return new AuthResponse(token, user.getEmail(), user.getRole(), "Login
    successful");
}

```

MemberService.java — all methods in plain English

<code>createMember(member)</code>	Sets status='ACTIVE' first, then calls repo.save() → INSERT INTO members(...)
<code>getAllMembers()</code>	Calls repo.findAll() → SELECT * FROM members. Returns a List.
<code>getMemberById(id)</code>	Calls repo.findById(id). Returns Optional. orElseThrow() gives 500 if not found.
<code>updateMember(id, updated)</code>	Finds existing member, copies the new values into it, calls repo.save() → UPDATE members SET...
<code>deleteMember(id)</code>	Does NOT delete the row. Sets status='INACTIVE' and saves. This is called a soft delete.

PaymentService.java — processPayment() key steps

```

public Payment processPayment(PaymentRequest req) {

    // Validate payment method is one of the allowed values
    List allowed = List.of("CREDIT_CARD", "DEBIT_CARD", "UPI", "QR_CODE", "CASH");
    if (!allowed.contains(req.getPaymentMethod())) throw new RuntimeException("Invalid
    method");

    // Generate a unique transaction ID using UUID
    String txnId = "TXN-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase();
    // e.g. TXN-A1B2C3D4

    // Set status to SUCCESS (dummy – real system would call Razorpay/Stripe here)
}

```

```
payment.setStatus("SUCCESS");
payment.setPaidAt(LocalDate.now());

Payment saved = repo.save(payment); // INSERT INTO payments(...)

// Send receipt email – inside try/catch so payment doesn't fail if email fails
try { notificationClient.sendNotification(...); }
catch (Exception e) { System.out.println("Email failed: " + e.getMessage()); }

return saved;
}
```

What is a Controller?

The controller is the front door of each service. It receives the HTTP request from the gateway, reads the data from it (body, URL, headers), calls the service, and returns the response as JSON.

MemberController.java — complete with explanation of every line

```
@RestController // = @Controller + @ResponseBody: return JSON automatically
@RequestMapping("/members") // all endpoints start with /members
public class MemberController {

    @Autowired
    private MemberService service; // Spring injects MemberService here

    @PostMapping // handles POST /members
    public Member create(@RequestBody Member member) {
        // @RequestBody: Spring converts incoming JSON → Member object automatically
        return service.createMember(member);
        // return value is converted back to JSON automatically
    }

    @GetMapping // handles GET /members
    public List getAll(@RequestHeader("X-User-Role") String role) {
        // @RequestHeader: reads the X-User-Role header gateway added
        if (!"ADMIN".equals(role)) throw new RuntimeException("Access denied");
        return service.getAllMembers();
    }

    @GetMapping("/{id}") // handles GET /members/1 or GET /members/5
    public Member getById(@PathVariable Long id) {
        // @PathVariable: extracts '1' or '5' from the URL
        return service.getMemberById(id);
    }

    @PutMapping("/{id}") // handles PUT /members/1
    public Member update(@PathVariable Long id, @RequestBody Member member) {
        return service.updateMember(id, member);
    }

    @DeleteMapping("/{id}") // handles DELETE /members/1
    public String delete(@PathVariable Long id,
        @RequestHeader("X-User-Role") String role) {
        if (!"ADMIN".equals(role)) throw new RuntimeException("Access denied");
    }
}
```



```

return service.deleteMember(id); // returns "Member deactivated"
}
}

```

PlanController.java — subscribe uses @RequestParam (URL params, not body)

```

// POST /plans/subscribe?memberId=1&memberEmail=abc@gmail.com&planId=2
@PostMapping("/subscribe")
public MemberSubscription subscribe(
    @RequestParam Long memberId, // reads ?memberId=1 from URL
    @RequestParam String memberEmail, // reads ?memberEmail=abc@gmail.com
    @RequestParam Long planId) { // reads ?planId=2
    return service.subscribe(memberId, memberEmail, planId);
}

```

@RestController	Returns JSON. Without this, Spring would try to return an HTML view file.
@RequestMapping("/members")	Base path. All methods inside inherit this prefix.
@PostMapping	Handle HTTP POST. Used for CREATE operations.
@GetMapping	Handle HTTP GET. Used for READ operations.
@PutMapping	Handle HTTP PUT. Used for UPDATE operations.
@DeleteMapping	Handle HTTP DELETE. Used for DELETE (but we soft-delete here).
@RequestBody	Reads the JSON from the request body and converts it to the Java object.
@PathVariable	Reads a value from the URL path. GET /members/5 → id=5.
@RequestHeader	Reads a specific HTTP header. Used to read X-User-Role that gateway added.
@RequestParam	Reads a query parameter from the URL. /subscribe?planId=2 → planId=2.

What is a DTO?

DTO = Data Transfer Object. It's a simple class that represents the shape of JSON coming IN or going OUT. We use DTOs instead of the entity directly because we often want to send/receive only some fields, not all of them.

DTOs in GymPro explained:

LoginRequest.java

Used in: auth-service — INPUT

Purpose: What Postman sends to POST /auth/login { "email": "admin@gympro.com", "password": "admin123" }

```
Fields: email, password
```

AuthResponse.java

Used in: auth-service — OUTPUT

Purpose: What auth-service returns after login/register { "token": "eyJ...", "email": "...", "role": "ADMIN", "message": "Login successful" }

```
Fields: token, email, role, message
```

BookingRequest.java

Used in: booking-service — INPUT

Purpose: What Postman sends to POST /bookings/create Contains memberId, memberEmail, trainerId, trainerEmail, scheduleId, sessionDay, sessionTime, notes

```
8 fields – all booking details
```

PaymentRequest.java

Used in: payment-service — INPUT

Purpose: What Postman sends to POST /payments/pay Contains memberId, memberEmail, bookingId OR subscriptionId, amount, paymentMethod, description

```
7 fields – payment details
```

Tip: The entity (like User.java) has ALL fields including password. The DTO (like AuthResponse) has only what we WANT to send back. We never return the password field in a response.

What is JWT?

JWT = JSON Web Token. It's a string the server creates after login. The client sends this string in every request as proof of identity. It has 3 parts separated by dots: Header.Payload.Signature

```
eyJhbGciOiJIUzI1NiJ9 ← Part 1: Header (base64) – algorithm used
.eyJzdWIiOiJhZG1pbkBNZW1wcm8uY29tIi... ← Part 2: Payload (base64) – email + role +
expiry
.xK8pZmL3_vQ2R1sT9... ← Part 3: Signature – proves it wasn't tampered
```

JwtUtil.java — generateToken() fully explained

```
@Component
public class JwtUtil {

    @Value("${jwt.secret}") // reads jwt.secret from application.properties
    private String secret;

    @Value("${jwt.expiration}") // reads jwt.expiration (86400000 = 24 hours)
    private long expiration;

    private Key getKey() {
        // Converts secret string to a 256-bit HMAC-SHA256 key
        return Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }

    public String generateToken(String email, String role) {
        return Jwts.builder()
            .setSubject(email) // who the token belongs to
            .claim("role", role) // embed role inside the token
            .setIssuedAt(new Date()) // when was it created
            .setExpiration(new Date(System.currentTimeMillis() + expiration))
            .signWith(getKey(), SignatureAlgorithm.HS256) // sign with secret
            .compact(); // build to string
    }
}
```

Why the secret must match in auth-service AND gateway-service:

```
# application.properties – both services must have IDENTICAL jwt.secret
# auth-service: creates (signs) the token using this secret
# gateway-service: validates (verifies) the token using the SAME secret

jwt.secret=gympro_jwt_secret_key_for_hs256_algorithm_minimum_256_bits
```

```
# If the secrets are different → gateway says 'invalid token' → 401 error
```

What is JwtAuthFilter?

This is a `GlobalFilter` in `gateway-service`. It runs before every request that comes into the gateway. It checks if the JWT token is valid. If yes, it extracts the user's email and role and adds them as headers before forwarding the request to the target service.

```
@Component
public class JwtAuthFilter implements GlobalFilter, Ordered {

    // These URLs don't need a token
    private static final List PUBLIC_URLS =
        List.of("/auth/register", "/auth/login", "/auth/test");

    @Override
    public Mono filter(ServerWebExchange exchange, GatewayFilterChain chain) {

        String path = exchange.getRequest().getURI().getPath();

        // Step 1: Is this a public URL? If yes, skip token check
        if (PUBLIC_URLS.stream().anyMatch(path::startsWith)) {
            return chain.filter(exchange); // pass through without token check
        }

        // Step 2: Read the Authorization header
        String authHeader = exchange.getRequest().getHeaders()
            .getFirst(HttpHeaders.AUTHORIZATION);

        // Step 3: Check format is 'Bearer eyJ...'
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete(); // → 401
        }

        try {
            // Step 4: Extract token (remove 'Bearer ' prefix – 7 characters)
            String token = authHeader.substring(7);

            // Step 5: Validate signature and parse claims
            Claims claims = Jwts.parserBuilder()
                .setSigningKey(Keys.hmacShaKeyFor(secret.getBytes(UTF_8)))
                .build()
                .parseClaimsJws(token) // throws exception if invalid/expired
                .getBody();

            // Step 6: Extract role from token payload
        }
    }
}
```

```

String role = claims.get("role", String.class);

// Step 7: Add user info as headers before forwarding to service
exchange = exchange.mutate().request(r -> r
    .header("X-User-Email", claims.getSubject()) // adds email header
    .header("X-User-Role", role) // adds role header
).build();

} catch (Exception e) {
    exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
    return exchange.getResponse().setComplete(); // → 401
}

return chain.filter(exchange); // forward to correct service
}

@Override
public int getOrder() { return -1; } // -1 = run before all other filters
}

```

Tip: This is WHY all other services use SecurityConfig with `.anyRequest().permitAll()` — the gateway already did the security check. Services trust the X-User-Role header that gateway added.

What is OpenFeign?

Without Feign, calling another microservice requires ~15 lines of RestTemplate code. With Feign, you write a Java interface and Spring creates the HTTP client automatically. When you call `notificationClient.sendNotification(...)` it looks like a normal Java method call but it actually makes an HTTP request to notification-service.

NotificationClient.java (in booking-service)

```
@FeignClient(name = "notification-service")
// name = the spring.application.name of the target service
// Feign asks Eureka: 'where is notification-service?' and gets the address
public interface NotificationClient {

    @PostMapping("/notify/send")
    // Feign will make: POST http://notification-service/notify/send
    String sendNotification(
        @RequestParam String email, // ?email=shreyas@gympro.com
        @RequestParam String subject, // &subject;=Session+Booked
        @RequestParam String message // &message;=Hi!+Your+session...
    );
}
```

How Feign is used in BookingService.java:

```
@Autowired private NotificationClient notificationClient;

// Booking saved to DB first
Booking saved = repo.save(booking);

// Then try to send email – inside try/catch so booking succeeds even if email fails
try {
    notificationClient.sendNotification(
        req.getMemberEmail(),
        "GymPro - Session Booked",
        "Your session on " + req.getSessionDay() + " is confirmed."
    );
    // This line ↑ looks like a normal method call
    // But it actually makes: POST
    http://notification-service/notify/send?email=...&subject;=...&message;=...
} catch (Exception e) {
    System.out.println("Email failed: " + e.getMessage());
}
```

```
// booking is NOT cancelled – email is a side effect  
}
```

Who uses Feign in GymPro:

booking-service	Has NotificationClient → calls notification-service to email member + trainer when booking is created or cancelled
payment-service	Has NotificationClient → calls notification-service to email receipt after payment or refund
notification-service	Has NO Feign client — it is the receiver, not the caller
@EnableFeignClients	Must be on the main class of booking-service and payment-service for Feign to work

Full Request Walkthrough

From Postman click to MySQL row — every step traced

Scenario: Member books a trainer session

Postman sends: POST `http://localhost:8080/bookings/create` with Bearer token and JSON body

Step 1	Postman	Sends POST <code>/bookings/create</code> to <code>localhost:8080</code> Headers: Authorization: Bearer <code>eyJhbGc...</code> Body: <code>{ memberId:1, trainerId:1, scheduleId:1, sessionDay:MON, sessionTime:09:00-11:00 }</code>
Step 2	gateway-service :8080	JwtAuthFilter runs: 1. Path <code>/bookings/create</code> is NOT in <code>PUBLIC_URLS</code> 2. Reads Authorization header 3. Extracts token: <code>authHeader.substring(7)</code> 4. Validates with <code>Jwts.parserBuilder()...parseClaimsJws()</code> 5. Extracts email and role from token claims 6. Adds X-User-Email and X-User-Role headers 7. Looks up routing rules: <code>/bookings/**</code> → <code>lb://booking-service</code> 8. Asks Eureka: where is booking-service? Gets <code>127.0.0.1:8085</code> 9. Forwards request to booking-service:8085
Step 3	booking-service :8082	<code>BookingController.create()</code> is called: <code>@RequestBody BookingRequest req</code> → Spring converts JSON to <code>BookingRequest</code> object Calls <code>service.createBooking(req)</code>
Step 4	BookingService	<code>createBooking(req)</code> runs: 1. Creates new <code>Booking</code> object, sets all fields from req 2. Sets <code>bookingDate = LocalDate.now()</code> 3. Sets <code>status = 'CONFIRMED'</code> 4. Calls <code>repo.save(booking)</code> → Hibernate runs: <code>INSERT INTO bookings(...) VALUES(...)</code>
Step 5	MySQL	Inserts row into <code>gympro_booking.bookings</code> table Auto-increments <code>id = 1</code> Returns the saved <code>Booking</code> object with <code>id</code> filled in
Step 6	BookingService	Saved booking returned. Now sends email: try { <code>notificationClient.sendNotification(memberEmail, ...)</code> } Feign asks Eureka: where is notification-service? Makes POST <code>http://notification-service/notify/send?email=...</code>
Step 7	notification-service :8086	<code>NotificationController.send()</code> called Calls <code>emailService.sendEmail()</code> <code>JavaMailSender</code> connects to <code>smtp.gmail.com:587</code> Sends email to member and trainer
Step 8	Response flows back	booking-service returns <code>Booking</code> JSON to gateway Gateway forwards response to Postman Postman shows: 200 OK + the booking JSON with <code>id</code>

Annotation	Where used	What it does
CLASS-LEVEL ANNOTATIONS		
@SpringBootApplication	All XxxApplication.java	= @Configuration + @EnableAutoConfiguration + @ComponentScan. St
@EnableEurekaServer	EurekaServerApplication	Makes this app a Eureka registry server at port 8761.
@EnableDiscoveryClient	All business services	Registers this service with Eureka on startup.
@EnableFeignClients	Booking + Payment	Scans for @FeignClient interfaces and creates HTTP proxy objects.
@Configuration	SecurityConfig	This class contains @Bean method definitions.
@EnableWebSecurity	SecurityConfig	Activates Spring Security and enables web security customization.
@RestController	All controllers	= @Controller + @ResponseBody. Every method returns JSON automati
@RequestMapping("/x")	All controllers	All methods in this class have /x as their base URL prefix.
@Component	JwtUtil, JwtAuthFilter	Generic Spring-managed bean — Spring creates one instance and mana
@Service	All service classes	Marks the business logic layer. Spring manages one instance.
@Entity	All entity classes	This Java class maps to a MySQL database table.
@Table(name='x')	All entity classes	Sets the exact MySQL table name.
FIELD-LEVEL ANNOTATIONS		
@Id	Entity id fields	Marks this field as the primary key of the table.
@GeneratedValue(IDENTITY)	Entity id fields	MySQL AUTO_INCREMENT — id is set by MySQL, not by you.
@Column(unique=true)	email fields	Adds UNIQUE constraint to this column in MySQL.
@Column(nullable=false)	Required fields	Adds NOT NULL constraint to this column in MySQL.
@Autowired	Service, controller fields	Dependency Injection — Spring finds and injects the matching bean.
@Value("\${key}")	JwtUtil fields	Reads a value from application.properties into this field.
LOMBOK ANNOTATIONS		
@Data	All entities + DTOs	Generates: getters, setters, toString, equals, hashCode.
@NoArgsConstructor	All entities	Generates empty constructor. JPA REQUIRES this to create objects from
@AllArgsConstructor	All entities + DTOs	Generates constructor with ALL fields as parameters.
METHOD-LEVEL ANNOTATIONS		
@Bean	SecurityConfig methods	Return value becomes a Spring-managed singleton object.
@PostMapping("/x")	Controller methods	Handle HTTP POST /x — typically for CREATE.
@GetMapping("/x")	Controller methods	Handle HTTP GET /x — typically for READ.
@PutMapping("/x")	Controller methods	Handle HTTP PUT /x — typically for UPDATE.
@DeleteMapping("/x")	Controller methods	Handle HTTP DELETE /x — typically for DELETE.
PARAMETER ANNOTATIONS		
@RequestBody	Controller params	Converts incoming JSON body to Java object automatically.

@PathVariable	Controller params	Extracts value from URL path. GET /members/{id} → id variable.
@RequestHeader	Controller params	Reads a specific HTTP header. Used to read X-User-Role from gateway.
@RequestParam	Controller params	Reads query param from URL. /subscribe?planId=2 → planId=2.
FEIGN ANNOTATIONS		
@FeignClient(name='x')	NotificationClient	Interface becomes an HTTP client that calls service named x via Eureka.
@PostMapping inside Feign	NotificationClient methods	Feign will make a POST request to this path on the target service.